SHIVALIK UNIVERSITY

DEPARTMENT: B.TECH CSE

PROJECT REPORT

HONEYSHIELD

Evaluating Honey-Token Enforcement at Tool-Call Boundaries in RAG-Based LLM Applications

Submitted by
Gourav Kumar
Nimesh Singh Rana

Academic Session: 2026–2027

Shivalik University

HONEYSHIELD

Evaluating Honey-Token Enforcement at Tool-Call Boundaries in RAG-Based LLM Applications

HONEYSHIELD

Evaluating Honey-Token Enforcement at Tool-Call Boundaries in RAG-Based LLM Applications

Fully documented project report — working draft for review

Implementation and experiments use synthetic canaries, synthetic test cases, and a harmless local mock tool.

Abstract

HoneyShield is a small, controlled security experiment for a retrieval-augmented generation (RAG)-style application that can request tool execution. It investigates whether inspecting structured tool-call arguments before dispatch, alongside filtering the final assistant response, prevents disclosure of a registered synthetic canary more effectively than output-only filtering or no canary guard. The study compares three configurations using paired model decisions, then evaluates several exact-match, normalization, punctuation-aware, and Base64-aware string guards on constructed positive and benign test cases. A local mock tool is used; no real credentials or external actions are involved.

In a 10-repetition paired live pilot, one shared model decision per scenario/repetition was replayed across the three configurations. Among valid decisions, the baseline disclosed canaries in 20/29 rows, output-only filtering in 10/29, and boundary-plus-output filtering in 0/29. In the tool-canary scenario, the boundary guard blocked 10/10 calls before mock dispatch; output-only filtering blocked final responses but did not stop the tool-call arguments from crossing the boundary. The included fresh-canary holdout-style CSV contains 26 full-positive cases, 31 benign cases, and 7 edge cases; the Base64-aware v3 candidate blocked 26/26 full positives and 0/31 benign cases in both text and argument inspection. A 50-case constructed positive stress set reported 50/50 detection and a 100-case constructed benign stress set reported 0/100 false positives. These are small, constructed evaluations, not evidence of universal protection or a real-world zero false-positive rate.

The defensible contribution is a reproducible, implementation-level comparison of enforcement placement and selected canary transformations under a paired decision design, with explicit measurement of detection, benign-call preservation, and guard-only latency. Boundary enforcement and canary-based detection are established ideas; HoneyShield does not claim to invent either.

1. Project Identity and Overview

2. Problem Statement

RAG applications retrieve external documents and place their contents into a model's context. Tool-using LLM applications may also let the model emit structured requests—such as a tool name and JSON arguments—that application code executes. If sensitive content or a synthetic canary is present in the context, a model may attempt to reproduce it in a final answer or place it inside tool arguments. A final-response filter can prevent a string from being shown to the user, but it cannot retroactively prevent an already-dispatched tool call from carrying that string.

2.1 Problem being addressed

- Final-output-only inspection may be too late to stop data from crossing a tool-call boundary.
- Exact string matching is simple and fast but can miss selected transformations such as whitespace insertion, punctuation changes, or Base64 encoding.
- More aggressive normalization may increase detection while also risking false positives, so security and benign utility must be measured together.
- Claims about a guard are difficult to interpret without explicit attack cases, benign cases, paired decisions, reproducible artifacts, and stated limitations.

2.2 Aim and objectives

1. Build a safe local harness with a harmless mock tool and registered synthetic canaries.
2. Compare no canary guard, final-output-only filtering, and pre-dispatch argument inspection plus final-output filtering.
3. Compare baseline exact matching with progressively more transformation-aware canary guards.
4. Measure detection coverage, false positives on constructed benign samples, benign tool-call preservation, and guard-only latency.
5. Document implementation, data artifacts, reproducibility steps, literature context, limitations, and future improvements.

2.3 Research questions

- RQ1: In the paired live pilot, how do the three enforcement configurations differ in synthetic-canary disclosure?
- RQ2: How do selected normalization and Base64-aware transformations affect canary detection on constructed test sets?
- RQ3: What guard-only latency overhead does the more comprehensive candidate add compared with baseline exact matching?
- RQ4: What can and cannot be concluded from the current controlled sample sizes and test construction?

3. Scope, Assumptions, and Threat Model

3.1 In scope

- Synthetic canaries generated specifically for the experiment.
- RAG-like context and structured tool-call arguments.
- Pre-dispatch and final-output inspection.
- Harmless local mock search tool and deterministic test harness.
- Selected exact, case, separator, punctuation, whitespace, and Base64 transformations.

3.2 Out of scope

- Real passwords, API keys, personal data, or production secrets.
- External network tools, shell execution, email sending, file deletion, or other side effects.
- A broad benchmark of indirect prompt injection across many models.
- Semantic paraphrase detection, arbitrary encodings, multi-turn covert channels, or proof of production readiness.
- A claim that canary matching alone solves prompt injection.

3.3 Threat model and definitions

This definition is deliberately narrow. A blocked tool request is not a successful boundary disclosure because the arguments do not cross the dispatch boundary. An attempted canary in a final answer is not a user-visible disclosure if the output guard blocks it.

4. Core Concepts: Glossary and Definitions

5. System Architecture and Data Flow

The implementation is a local experiment harness rather than a production RAG platform. The central comparison is the placement of the guard around a structured tool call and final response.

6. Create and register a synthetic canary.
7. Build a test prompt and context; ask the local model for either a structured tool-call decision or a final response.
8. Parse the model decision into a known schema. Invalid decisions are recorded as invalid rather than silently treated as benign.
9. Configuration A (baseline): no canary-specific boundary or output guard; the mock tool is still allowlisted.
10. Configuration B (output-only): permit the tool-call arguments to reach the mock dispatch boundary, then inspect the final response before release.
11. Configuration C (boundary + output): inspect serialized/structured tool arguments before dispatch, block canary-bearing requests, and inspect final output before release.
12. Write per-case results to CSV, summarize metrics, run regression tests, and hash selected artifacts for the reproducibility manifest.

Architecture in words: User/test scenario → prompt/context assembly → local LLM → decision parser → [pre-dispatch argument guard in C] → allowlisted mock tool → tool result/context loop → final response guard in B and C → released answer. A lacks canary guards; B only checks final output; C checks both boundary and final output.

6. Implementation and Code Walkthrough

6.1 Guard logic, conceptually

The baseline guard serializes JSON-compatible tool arguments and searches for the exact registered canary string. The more comprehensive candidate checks text case-insensitively while tolerating selected separators, scans plausible Base64 tokens, decodes eligible UTF-8 payloads, and recursively inspects strings in dictionaries and lists. Invalid/non-JSON-compatible tool arguments are blocked by the argument guard rather than passed through.

The guard is a deterministic string inspection rule, not a semantic model. It does not understand whether an action is authorized in context, and it does not guarantee detection of paraphrases, arbitrary encodings, split information across independent calls, compressed/encrypted payloads, or multi-turn exfiltration.

6.2 Safety and dispatch controls

- Only the named harmless mock tool is allowlisted.
- No real credentials are stored; canaries are synthetic.
- The demo includes benign input, exact canary input, and Base64 canary input.
- Assertions check expected dispatch/block outcomes.
- The LLM pilot is separate from deterministic guard-only evaluations.

7. Experimental Methodology

7.1 Experiment families

7.2 Paired live pilot scenarios

- tool_canary: model proposes a tool call whose argument contains the canary.

- final_canary: model's proposed final answer contains the canary.
- benign: model proposes a harmless benign request, expected to remain usable.

The same model decision was replayed across A, B, and C for each scenario/repetition. This pairing reduces variation caused by different model generations across configurations. One benign model decision was invalid in the 10-repetition pilot, yielding 29 valid decisions per configuration rather than 30. Invalid decisions should remain visible in reporting and not be relabelled as successes.

8. Results

8.1 Paired live pilot (10 repetitions)

Interpretation: in the tested tool-canary scenario, A allowed all 10 canary-bearing tool calls to cross the mock dispatch boundary; B also allowed those calls to cross the boundary, although the final output filter blocked the final response; C blocked all 10 canary-bearing calls before dispatch. For final-canary scenarios, B and C blocked all 10 outputs. In benign scenarios, all configurations dispatched 9/9 valid benign calls. One invalid benign model decision was shared across configurations.

The aggregate disclosure counts are 20/29 for A, 10/29 for B, and 0/29 for C. These aggregate counts combine different scenario types and should be interpreted alongside the scenario breakdown. They are not a population estimate or a benchmark against all prompt-injection attacks.

8.2 Fresh-canary holdout-style guard evaluation

The included holdout-style CSV contains 64 cases: 26 full-positive, 31 benign, 4 partial-token, and 3 long-identifier cases. The v3 candidate blocked 26/26 full positives and 0/31 benign cases in both text and tool-argument inspection. It allowed all 7 cases labelled partial or long_identifier. The dataset is a constructed challenge set with a fresh canary, not a production benchmark; the zero observed false positives apply only to these 31 benign cases.

8.3 Positive and benign stress tests

These stress sets are constructed and are not independent, real-world attack corpora. They are useful regression and coverage checks, not broad claims of adversarial robustness.

8.4 Guard-only latency

The benchmark ran 5,000 iterations per guard/case after warm-up. Measurements exclude Ollama inference, parsing overhead outside the guard, and tool execution. Results are local microbenchmarks and may vary by machine, Python runtime, input length, and implementation. The extra few hundredths of a millisecond observed here are small in absolute terms, but this does not measure end-to-end latency.

8.5 Reproducibility and tests

- Latest recorded unit-test run: 16/16 passing using `python -m unittest discover -s tests -v`.
- `pytest` was not installed in the environment; use the standard-library unittest command above.
- A reproducibility manifest records selected CSV row/column counts and SHA-256 hashes.
- The manifest checks that artifacts exist and are readable; it does not independently recompute every metric.
- Live model results can vary with model version, prompt, runtime, decoding parameters, and machine state. Record exact environment and rerun artifacts before final submission.

9. Literature Review

The literature shows that canary-based prompt-leak detection, deception tokens, indirect prompt-injection defenses, and runtime tool-call enforcement are established research directions. HoneyShield should therefore make a narrow comparative contribution rather than claim that honey-tokens or tool-boundary checking are new.

9.1 Canary-based prompt-leak detection

Gakh and Bahsi evaluate early detection systems including LLM Guard, Vigil, and Rebuff against prompt-leak attacks. Their work compares detection techniques, including canary-word checks, and reports weaknesses in existing canary implementations, proposing improvements. This is the closest direct methodological neighbor for canary detection and false-positive/detection trade-offs. HoneyShield differs in its small paired three-configuration comparison of where a check is enforced—tool arguments before dispatch versus final output—and in its local transformation-focused guard progression. The overlap means the project must not claim to be the first canary detector or first to improve canary matching. [1]

The 2025 USENIX Security paper “Cloak, Honey, Trap: Proactive Defenses Against LLM Agents” explores deception, including LLM-specific honey-tokens, to expose or disrupt malicious agents. It establishes honey-token/deception approaches as a broader agent-security strategy. HoneyShield uses a simpler synthetic canary as a measurable marker and does not implement its full deception or counterattack framework. [2]

9.2 Tool-call and runtime enforcement

ClawGuard describes a runtime security framework for tool-augmented agents that checks policy at tool-call boundaries and blocks unsafe actions before real-world effects. This is direct overlap with the placement of a guard before dispatch; boundary enforcement itself is not novel. HoneyShield's narrower scope is to compare three configurations with shared model decisions and measure canary movement specifically. [3]

SecureClaw proposes dual-boundary protection: sensitive reads are confined through a trusted gateway, and writes are authorized at the effect sink. It emphasizes that protecting only one boundary may leave another exposure path. HoneyShield also uses multiple enforcement points, but it is much simpler: string/encoding inspection of arguments and final responses, without opaque handles, a declassification interface, or a transactional preview/commit protocol. [4]

ActGuard audits proposed actions before execution and evaluates whether tool choices or parameters deviate from locally expected behavior. This is a richer action-reasoning approach than HoneyShield's deterministic canary matching. It highlights a limitation of the present project: a canary guard only detects registered markers and cannot judge whether an otherwise canary-free tool action is unauthorized. [5]

9.3 Task alignment and prompt-injection defenses

The Task Shield reframes indirect prompt-injection defense around whether instructions and tool calls align with the user's goal. Its task-alignment method is orthogonal to canary matching: it can reason about an action's relevance even when no known secret string is present. HoneyShield does not evaluate task alignment and should be presented as a complementary, narrow signal rather than a replacement for semantic or policy-based defenses. [6]

9.4 Emerging canary/provenance fusion work

The 2026 HoneyWire working paper describes adaptive deception canaries combined with attention/provenance signals and a pre-execution gate. Its described approach is conceptually close to using canaries and inspecting pending structured actions before execution. It reports strong results, but those results belong to that paper's own evaluation setup and should not be directly compared numerically with HoneyShield's small synthetic tests. HoneyShield does not implement adaptive placement, provenance tracing, attention analysis, or benchmark-scale testing. Its useful contrast is a transparent, simple baseline that can be implemented and measured locally. [7]

9.5 Literature synthesis

9.6 References

[1] V. Gakh and H. Bahsi, “Enhancing security in LLM applications: a performance evaluation of early detection systems,” International Journal of Information Security, published online 3 Oct. 2026. https://doi.org/10.1007/s10207-026-01338-7 (preprint: https://arxiv.org/abs/2506.19109).

[2] D. Ayzenshteyn, R. Weiss, and Y. Mirsky, “Cloak, Honey, Trap: Proactive Defenses Against LLM Agents,” USENIX Security Symposium 2025, pp. 8095–8114. https://www.usenix.org/conference/usenixsecurity25/presentation/ayzenshteyn

[3] W. Zhao, Z. Li, P. Zhang, and J. Sun, “ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection,” arXiv:2604.11790, 2026. https://arxiv.org/abs/2604.11790

[4] Y. Ma and S. Schmid, “SecureClaw: Clawing Back Control of LLM Agents,” arXiv:2606.09549, 2026. https://arxiv.org/abs/2606.09549

[5] B. Wang et al., “ActGuard: Pre-execution Action Auditing against Indirect Prompt Injection in LLM Agents,” arXiv:2609.14987, 2026. https://arxiv.org/abs/2609.14987

[6] F. Jia, T. Wu, X. Qin, and A. Squicciarini, “The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents,” Proceedings of ACL 2025, pp. 29680–29697. https://doi.org/10.18653/v1/2025.acl-long.1435

[7] I. Ferreira, O. Haddad, and M. Reinhardt, “HoneyWire: A Case for Deception over Filtering — Adaptive Deception Canaries and Attention–Provenance Fusion for Runtime Detection of Indirect Prompt Injection in Tool-Using LLM Agents,” working paper / project publication page, 2026. https://tu-eisenfeld.de/en/library/publications/honeywire-prompt-injection-detection ; https://doi.org/10.5281/zenodo.22839414

10. Novelty and Research Contribution

10.1 What is not novel

- Honey-tokens/canaries as a general idea.
- Prompt-leak detection using known marker strings.
- Inspecting tool calls before execution as a general security pattern.
- Layered enforcement at more than one boundary.
- Normalizing strings or decoding Base64 as general programming techniques.

10.2 Defensible contribution

HoneyShield's contribution is an implementation-level, controlled comparison of three enforcement configurations using the same underlying model decisions across conditions, paired with a documented progression of canary-string inspection strategies and explicit reporting of benign preservation and guard-only latency. It demonstrates the specific failure mode where output-only filtering may block the final answer but does not prevent a canary-bearing tool argument from crossing the dispatch boundary. It also makes the evaluation limits visible by separating live model decisions from constructed deterministic guard test sets.

10.3 Suggested novelty statement for a report/viva

“We do not claim to introduce honey-tokens or tool-call boundary enforcement. HoneyShield contributes a small, reproducible implementation-level evaluation of where synthetic-canary checks are applied in a RAG-style tool-using LLM workflow. Using paired model decisions, we compare no canary guard, output-only filtering, and boundary-plus-output filtering, and evaluate selected transformations, benign-call preservation, and guard-only latency. The findings support the value of pre-dispatch inspection for the specific canary-bearing calls tested, while remaining limited by constructed datasets, one local model setup, and a narrow string-based threat model.”

11. Critical Questions and Suggested Viva Answers

What problem does HoneyShield solve?

It studies whether a canary-bearing tool request should be blocked before dispatch instead of relying only on a final-response filter.

Why is output filtering insufficient?

It only sees the final response. A tool call may already have carried the canary across a boundary before the final answer is generated.

What is RAG?

A system retrieves relevant records and places them in model context to help answer a query.

Why use synthetic canaries?

They provide a known marker for measuring unintended movement without using real secrets.

Does a canary in context mean a leak?

No. Context exposure is not counted as a leak. In this experiment a leak is a canary in a released answer or in tool arguments that cross mock dispatch.

Why use three configurations?

They isolate the contribution of boundary inspection from final-output filtering.

Why pair model decisions?

The same model decision is replayed across configurations, reducing generation-to-generation variability in comparisons.

What did the paired pilot show?

Among 29 valid rows per configuration, disclosures were 20 for baseline, 10 for output-only, and 0 for boundary-plus-output. In the tool-canary scenario C blocked all 10 before dispatch.

Did the model make mistakes?

One benign decision was invalid in the 10-repetition pilot and was shared across configurations. Invalid rows are reported rather than counted as benign successes.

Why Base64-aware detection?

A canary can be encoded as Base64 and therefore no longer appear as the original literal string. The guard decodes eligible Base64 tokens and checks the decoded text.

Is Base64 encryption?

No. It is an encoding; anyone can decode it without a secret key.

Can v3 detect every attack?

No. It handles selected transformations. It does not guarantee detection of paraphrases, arbitrary encodings, encryption, split multi-call channels, or canary-free malicious actions.

What is a false positive?

A benign input incorrectly blocked by the guard.

Can you claim a 0% false-positive rate?

Only for the tested constructed samples. Zero blocks in 31 or 100 benign examples does not establish a real-world 0% rate.

What is the limitation of the stress tests?

They were constructed by the project and are not a large independent real-world attack benchmark; some patterns also influenced development.

What does the latency benchmark include?

Only guard computation. It excludes Ollama inference, the rest of application parsing, and tool execution.

Is the project production-ready?

No. It is a controlled prototype. Production use needs stronger policies, independent benchmarks, audit logs, secret handling, parser hardening, and broader tests.

What is the novelty?

Not the general idea of canaries or boundary checks; the narrow contribution is the paired implementation-level comparison plus transformation, benign-preservation, and guard-latency measurements.

How is it different from semantic action auditing?

HoneyShield checks registered strings/encodings. Semantic auditing asks whether the proposed action fits the user's goal or policy.

What would you improve next?

Use a larger independent holdout, multiple models, confidence intervals, standard attack benchmarks, stronger tool policy, provenance tracking, and end-to-end latency/utility measurements.

Why is a mock tool used?

It allows dispatch-boundary behavior to be tested without external side effects or risk.

What evidence supports reproducibility?

Versioned scripts and CSV outputs, unit tests, a manifest with hashes and dimensions, and documented rerun commands; the manifest itself does not recalculate every metric.

12. Limitations, Validity Threats, and Ethics

- Single local model family and local runtime; results may not generalize to hosted models or other agent frameworks.
- Small paired pilot (10 repetitions) with one invalid benign model decision.
- Constructed transformation, positive-stress, and benign-stress samples; no independent large benchmark.
- Guard candidate evolved after inspecting failure cases, so the development-set 100% result is in-sample. Fresh-canary holdout-style evaluation is more informative but still small and constructed.
- Exact/normalized string guards do not understand authorization or semantic intent.
- False-positive estimates are uncertain at these sample sizes; zero observed false positives is not a zero population FPR.
- Guard-only latency excludes model inference and end-to-end tool path.
- Model output is stochastic and environment-sensitive; preserve model tag, prompts, settings, Python version, Ollama version, and source hashes in the final reproducibility record.
- Ethical safeguards: synthetic markers, harmless mock tool, no real secrets, no external side effects.

13. Future Work

13. Freeze guard implementation before defining a new independent holdout set; avoid tuning on test cases.
14. Evaluate multiple local and hosted models with recorded model/version, temperature, seeds where supported, and prompt templates.
15. Add a benchmark containing indirect prompt injection and realistic benign tool arguments, with independent annotation.
16. Report confidence intervals and per-scenario metrics; distinguish attack attempts, blocked actions, released leaks, and invalid model decisions.
17. Add strict schema validation, tool allowlists, argument type/range validation, authorization checks, and per-tool policies.
18. Explore provenance-aware policies and task-alignment checks as complementary controls, not replacements for deterministic canary checks.
19. Test nested and malformed encodings, Unicode confusables, percent encoding, HTML/entity encoding, and multi-turn aggregation in a safe harness.
20. Measure end-to-end latency and benign task utility, not just guard execution time.
21. Publish exact commands, environment details, dependency versions, prompts, CSV schemas, and a single rerun script.

Artifact verification note (25 Oct 2026): The uploaded HoneyShield source archive was inspected. The 90-row paired live pilot, 64-row holdout-style evaluation, 100-case benign stress set, 50-case positive stress set, 8-row latency benchmark, and 50-row encoded-guard evaluation were present. The reproducibility manifest row counts, column counts, and SHA-256 hashes matched the six listed CSVs. The unit test suite passed 16/16 tests, Python compilation passed, and the local mock-tool demo passed. README.md and requirements.txt in the uploaded archive are empty; see the reproducibility notes and setup limitations.

14. Reproduction Guide

Run from the project root with the virtual environment activated. Commands below assume the documented local project layout; confirm filenames exist before a final submission.

```
python -m unittest discover -s tests -v
python app/paired_live_pilot_v3_10rep.py
python app/holdout_guard_evaluation_v2.py
python app/benign_stress_evaluation.py
python app/positive_stress_evaluation.py
python app/guard_latency_benchmark.py
python app/final_reproducibility_audit.py
```

Important: the live pilot calls the local Ollama model and can take time. Re-running a script may overwrite its CSV output. Back up results before reruns and record the exact date, model tag, software versions, and hash manifest for the final report.

15. Conclusion

HoneyShield demonstrates, within its controlled setup, why inspection should occur before a tool call is dispatched when the security goal is to prevent a registered canary from crossing that boundary. Output-only filtering reduces released final-answer disclosure but does not prevent canary-bearing arguments from reaching the mock tool. The current boundary-plus-output configuration blocked all tested canary-bearing tool calls in the 10-repetition paired pilot and the v3 candidate detected all full positive cases in the current constructed fresh-canary holdout-style set. The results are promising but limited by small, constructed datasets, one local model, and a narrow string-based detector. The defensible conclusion is that boundary inspection adds a measurable enforcement layer for the tested canary cases—not that HoneyShield solves prompt injection generally.

Appendix A. Suggested figure to add before submission

Create a simple architecture diagram with the flow: Test/user prompt + retrieved synthetic context → local LLM → structured decision parser → (C only: argument guard) → allowlisted mock tool → tool result → final response → (B and C: output guard) → released answer. Mark blocked paths before dispatch and before release. Label A/B/C differences clearly.

Appendix B. Submission checklist

- Confirm university cover-page formatting, student details, department, and institution requirements.
- Verify all source citations and bibliographic details against publisher pages before submission.
- Attach final source-code repository URL and tag/commit hash.
- Rerun unit tests and all selected experiments from a clean environment; back up CSV outputs.
- Recompute metrics from CSV files and independently verify all counts in this report.
- Include architecture and sequence diagrams, sample CSV rows, and selected code excerpts.
- Keep limitations and novelty statement consistent across report, PPT, and viva answers.

Field	Description
Project name	HoneyShield

Full title	Evaluating Honey-Token Enforcement at Tool-Call Boundaries in RAG-Based LLM Applications
Project type	Implementation-oriented cybersecurity / AI application security experiment
Core question	Does pre-dispatch inspection of tool-call arguments plus final-output filtering reduce synthetic-canary disclosure compared with output-only filtering and no canary guard?
System under test	Local Ollama model (qwen2.5:3b) producing structured tool-call or final-answer decisions; harmless local mock tool
Safety boundary	Synthetic canary strings and synthetic cases only; allowlisted mock tool; no real secrets or external side effects
Primary outcomes	Canary disclosure, boundary blocks, output blocks, tool dispatches, benign-call preservation, transformation detection, and guard-only latency
Term	Meaning in this project
Asset	A registered synthetic canary string placed in test context.
Adversary action	A test prompt or model decision attempts to place the canary in a final response or tool argument.
Tool boundary	The application-code point after parsing a tool request but before dispatching it to the tool.
Successful disclosure	The registered canary is present in a released final answer or in arguments that cross the mock dispatch boundary.
Exposure in context	The model can see a canary; this alone is not counted as a leak.
Trust assumptions	The guard, registry, parser, and mock dispatcher run as intended and are not compromised.
Term	Beginner-friendly definition
Artificial intelligence (AI)	A broad field concerned with systems performing tasks associated with perception, reasoning, prediction, or language.
Large language model (LLM)	A model trained to predict and generate sequences of tokens; it can produce prose and structured outputs.
Token	A unit used by a language model to represent text; it may be a word, word fragment, punctuation, or other symbol.
Prompt	The input instructions and context supplied to a model for a particular generation.
System prompt	High-priority instructions supplied by the application to shape the model's behavior.
Prompt injection	An attempt to manipulate a model by placing instructions in input or context that conflict with the application's intended behavior.
Indirect prompt injection	Instructions embedded in retrieved or otherwise external content rather than only directly in the user's message.
Prompt leakage	Disclosure of instructions or content that the application intended to keep confidential.
Retrieval-augmented generation (RAG)	An architecture that retrieves relevant external records and includes them in model context to ground generation.
Retriever	A component that searches a document collection for records relevant to a query.
Embedding	A numeric vector representation of text or another item used to compare semantic similarity.
Vector database	A store optimized for indexing and searching vectors, often used for semantic retrieval.
Context	The information available to the model during a generation, including instructions, user text, and retrieved content.

Agent	An LLM-based system that selects steps or tools to pursue a goal.
Tool	A callable capability exposed to the model, such as search, database lookup, or a mock function.
Tool call	A structured request identifying a tool and supplying its arguments.
Tool-call arguments	The structured parameters sent to the selected tool, often represented as JSON.
Tool-call boundary	The enforcement point between a model's proposed call and actual tool dispatch.
Dispatch	The application action that passes a validated request to the tool implementation.
Output filtering	Inspection of the model's final response before the application releases it to the user or caller.
Canary / honey-token	A synthetic, uniquely identifiable marker placed in test context to reveal attempted or successful movement of that marker.
Synthetic canary	A test-only canary created for the experiment, not a real credential or secret.
Exact-match detection	A rule that detects a registered string when that exact string appears in inspected text.
Normalization	A transformation applied before comparison, such as case folding or removing separators.
Punctuation-aware matching	Matching that tolerates punctuation or separator changes between characters.
Base64	A binary-to-text encoding that represents bytes using a limited printable alphabet; it is not encryption.
Base64-aware detection	A guard that looks for Base64-looking tokens, decodes eligible tokens, and checks decoded text for a registered canary.
False positive (FP)	A benign input incorrectly blocked by a guard.
False negative (FN)	A positive canary-bearing input not detected by a guard.
True positive (TP)	A canary-bearing input correctly detected.
True negative (TN)	A benign input correctly allowed.
Detection rate / recall	TP / (TP + FN), the fraction of positive cases detected.
False-positive rate (FPR)	FP / (FP + TN), the fraction of benign cases incorrectly blocked.
Precision	TP / (TP + FP), the fraction of flagged cases that are positive in the labelled test set.
Utility / benign-call preservation	Whether legitimate test requests continue to be dispatched rather than blocked.
Latency	Elapsed time required for a computation; this project separately measures guard-only latency.
Median	The middle observation after sorting; less sensitive to extreme values than the mean.
95th percentile (p95)	A value at or below which 95% of measured observations fall.
Paired experiment	An experiment where the same underlying model decision or test case is evaluated across multiple configurations.
Reproducibility	The ability to rerun the documented procedure and obtain auditable outputs from stated code, data, and settings.
Allowlist	An explicit set of tool names that the dispatcher permits.
Mock tool	A harmless test implementation that simulates a tool without performing a real external action.
RCE / side effect	Remote code execution / a state-changing action; neither is used in this experiment.

Configuration	Boundary check	Final output check	Purpose
A_baseline	No canary guard	No canary guard	Reference condition
B_output_only	No canary guard	Yes	Tests the limitation of final-only filtering
C_boundary_output	Yes, before dispatch	Yes	Tests layered enforcement

File / module	Role
app/canaries.py	Generates synthetic canaries and supports basic containment checks.
app/guard.py	Baseline exact-match inspection for text and JSON-compatible tool arguments.
app/normalized_guard.py	Explores whitespace normalization and recursive argument inspection.
app/punctuation_guard.py	Adds alphanumeric normalization to tolerate punctuation/separator changes.
app/encoded_guard.py and app/encoded_guard_v2.py	Experimental Base64-aware candidates and refinements.
app/encoded_guard_v3.py	Current candidate: case/separator-aware matching plus bounded Base64 token decoding and recursive inspection of structured arguments.
app/mock_tools.py	Allowlisted harmless mock search tool; baseline and v3 guarded dispatch paths.
app/tool_agent.py	Calls local Ollama model and parses structured final/tool-call decisions.
app/paired_live_pilot_v3_10rep.py	Runs a paired live pilot: one model decision per scenario/repetition is replayed across A/B/C.
app/holdout_guard_evaluation_v2.py	Evaluates guard candidates on a fresh-canary constructed holdout-style set.
app/benign_stress_evaluation.py	Constructs benign stress cases to check accidental blocking.
app/positive_stress_evaluation.py	Constructs canary-bearing transformation cases to test detection.
app/guard_latency_benchmark.py	Measures guard-only latency, excluding model inference and tool execution.
app/final_reproducibility_audit.py	Checks selected result artifacts and records row/column counts and SHA-256 hashes.
tests/test_guard.py and tests/test_guard_v3.py	Unit tests for baseline and v3 behavior.

Experiment	Design	What it tells us
Paired live pilot	10 repetitions × 3 scenarios × 3 configurations = 90 rows; one model decision per scenario/repetition replayed across A/B/C	Comparative enforcement placement under shared decisions
Guard candidate evaluations	Constructed positive/benign examples; fresh canary for holdout-style run	Coverage against selected transformations
Benign stress test	100 constructed benign cases	Observed accidental blocking on this sample
Positive stress test	50 constructed canary-bearing cases	Observed detection on this sample
Latency benchmark	5,000 timed iterations per guard/case after warm-up; guard only	Local computational overhead excluding LLM/tool time
Unit tests	16 tests passing in latest recorded run	Regression confidence for implemented expected behavior

Configuration	Valid decisions	Canary disclosures	Boundary blocks	Output blocks	Tool dispatches
A_baseline	29/30	20	0	0	19
B_output_only	29/30	10	0	20	19
C_boundary_output	29/30	0	10	10	9

Guard candidate	Full positive cases detected	Detection rate	Benign cases blocked
Baseline exact text	11/26	42.3%	0/31
Normalized v2 text	14/26	53.8%	0/31
Punctuation v1 text	15/26	57.7%	0/31
Base64 v1 text	17/26	65.4%	0/31
Base64 v2 text	19/26	73.1%	0/31
Base64 v3 text	26/26	100%	0/31
Base64 v3 arguments	26/26	100%	0/31

Test set	Size	Result	Interpretation
Positive stress	50 constructed positive cases	50/50 detected for text and arguments	Good coverage for the tested generated patterns only
Benign stress	100 constructed benign cases	0/100 blocked for text and arguments	No false positives observed in this constructed sample only
Input case	Baseline median (approx.)	v3 median (approx.)	Scope
Short benign text	0.002 ms	0.017 ms	Guard only
Nested benign arguments	0.0025 ms	0.047 ms	Guard only
Exact canary	0.002 ms	0.015 ms	Guard only
Base64 canary	0.002 ms	0.021 ms	Guard only

Research direction	Established contribution	HoneyShield's distinction / limitation
Canary-based prompt-leak detection	Detection scanners, canary checks, evasion analysis and improved matching	Small local guard progression and paired comparison of enforcement placement; substantial overlap
Honey-tokens / deception	Canary-like markers and deception to expose malicious agents	Uses registered synthetic markers as a narrow measurement instrument; no active counterattack
Runtime tool boundary enforcement	Policy checks before tools execute	Compares a canary-specific boundary check against output-only filtering; does not invent boundary enforcement
Dual-boundary security	Protection at read and effect/write boundaries	Checks tool arguments and final response, but not sensitive-read confinement or side-effect authorization
Task-alignment / action auditing	Semantic checking of whether proposed actions fit the task	No semantic reasoning; canary-only detection misses attacks without the marker
Canary + provenance fusion	Adaptive canaries, provenance signals, pre-execution gating	No provenance fusion; emphasizes a simpler reproducible local experiment